

Making Hash Parallel, Thread-Safe and Fast!

Benoit Daloze

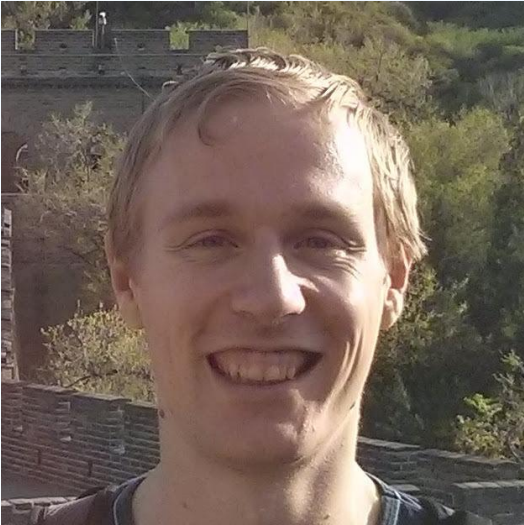


RubyKaigi 2026



DATADOG

Who am I?



Benoit Daloze

GitHub: @eregon

Website: <https://eregon.me>

Bluesky: @eregon.me

Mastodon:
@eregon@ruby.social

PhD on parallelism in dynamic languages

Maintainer of:

- TruffleRuby
- ruby/setup-ruby
- ruby/spec

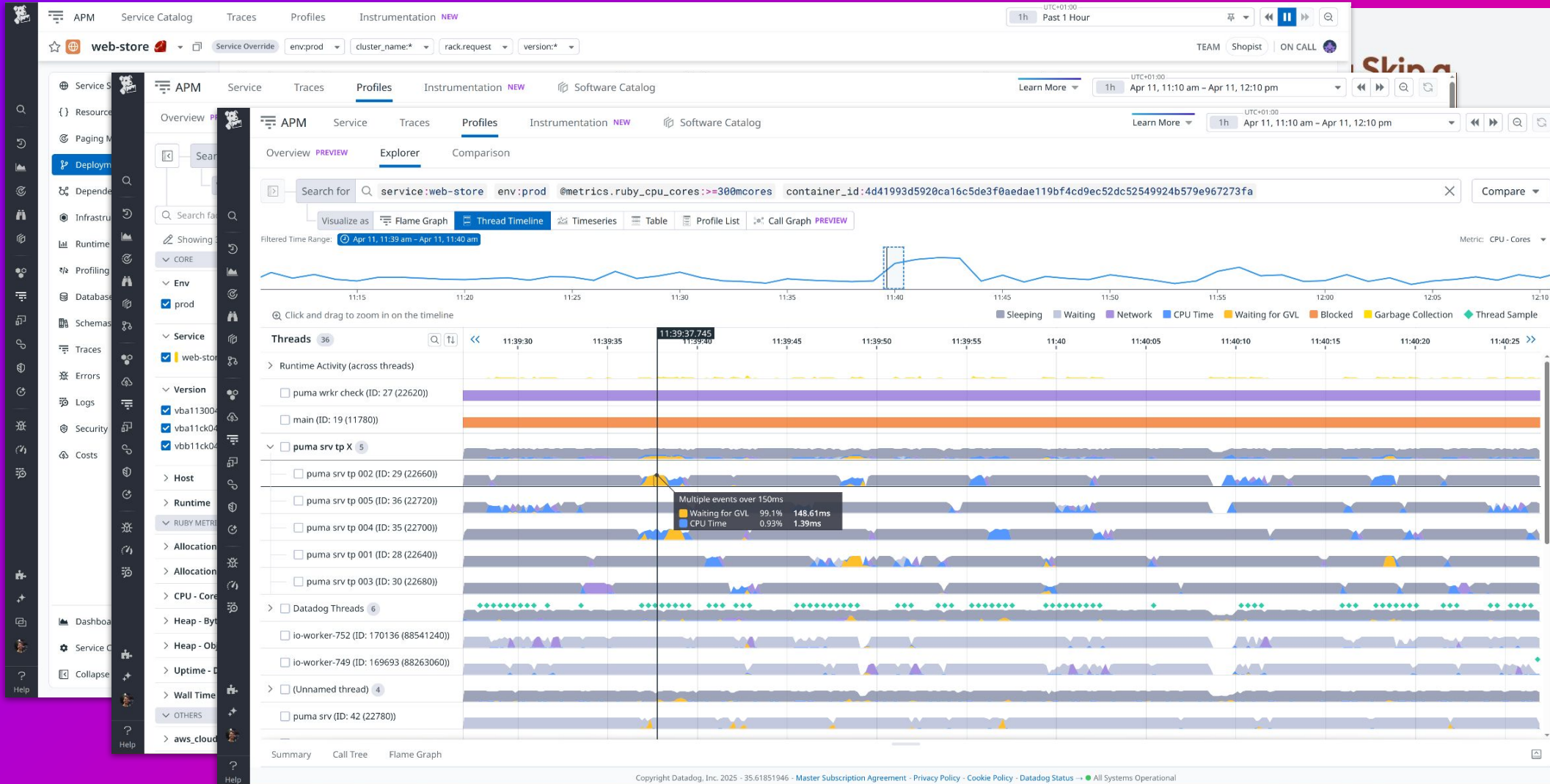
CRuby committer

Now working at Datadog on the Ruby Continuous Profiler!



Modern monitoring and security as a Service

Including profiling, distributed tracing, test optimization, security, errors, logs for Ruby



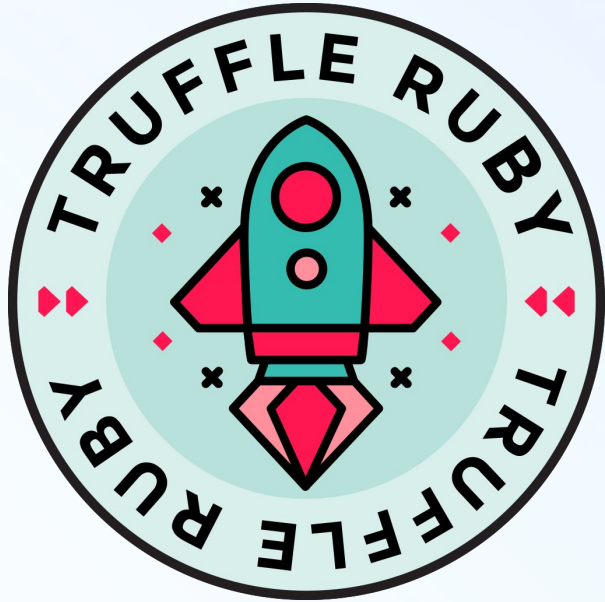
The Challenge

How to make Hash: { “Ruby” => “Kaigi!” }

- Thread-Safe
- Support parallel reads and writes
- Fast, no single-threaded overhead

Making Hash Parallel, Thread-Safe and Fast in ...

TruffleRuby



GitHub: [truffleruby/truffleruby](https://github.com/truffleruby/truffleruby)

Website: <https://truffleruby.dev>

Bluesky: [@truffleruby.dev](https://bsky.app/profile/@truffleruby.dev)

Mastodon:
[@truffleruby@ruby.social](https://mstdn.social/@truffleruby)

A high-performance Ruby implementation

No Global VM Lock: TruffleRuby runs Threads, Ruby code & native extensions in parallel!

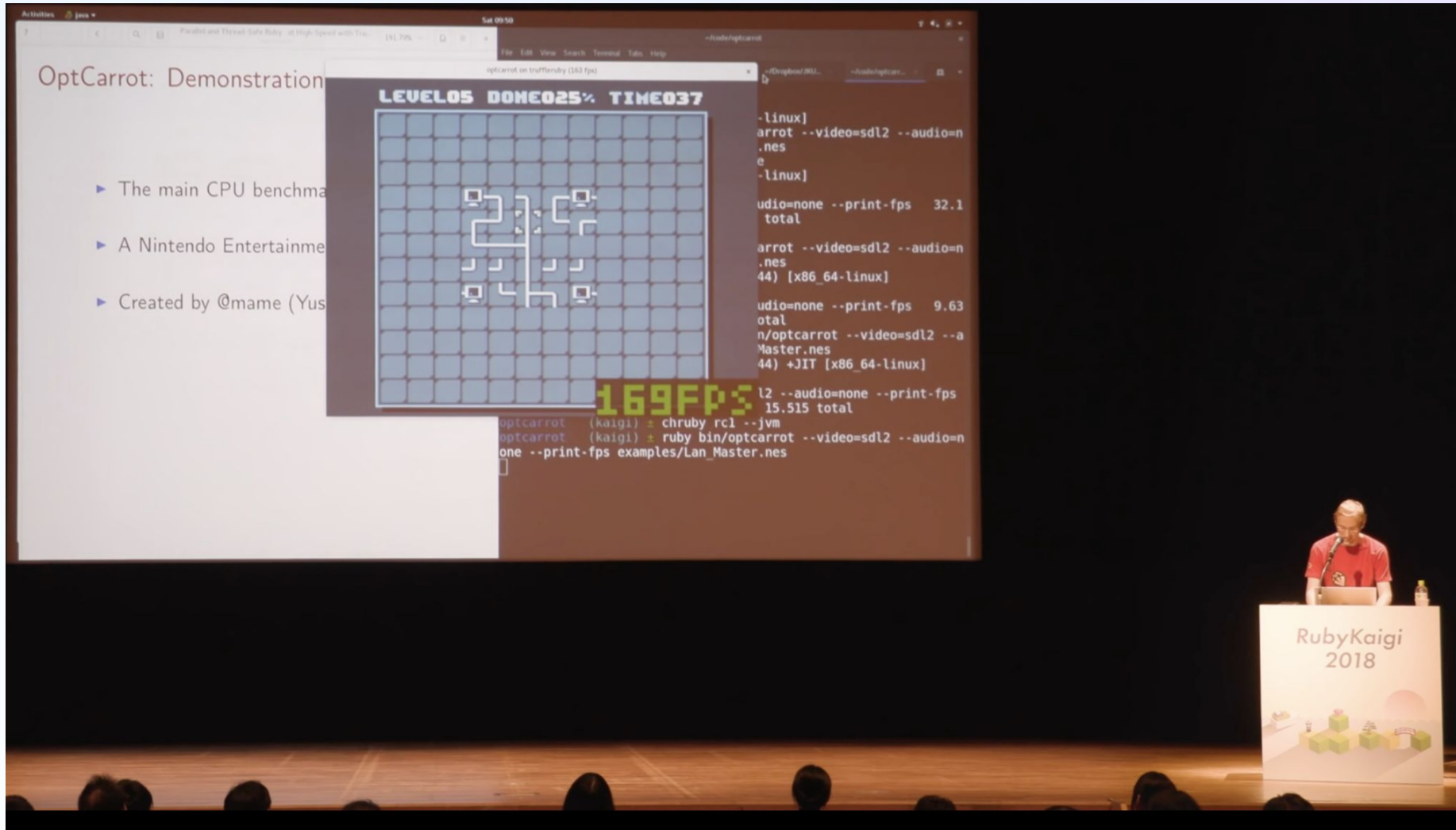
Supports native extensions and most popular gems

Passes 97% of ruby/spec

TruffleRuby uses the GraalVM JIT Compiler

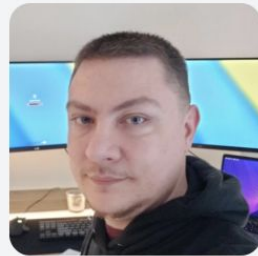
But how fast is it?

Parallel and Thread-Safe Ruby at High-Speed with TruffleRuby at RubyKaigi 2018



Demo Time!

Bonus Demo!

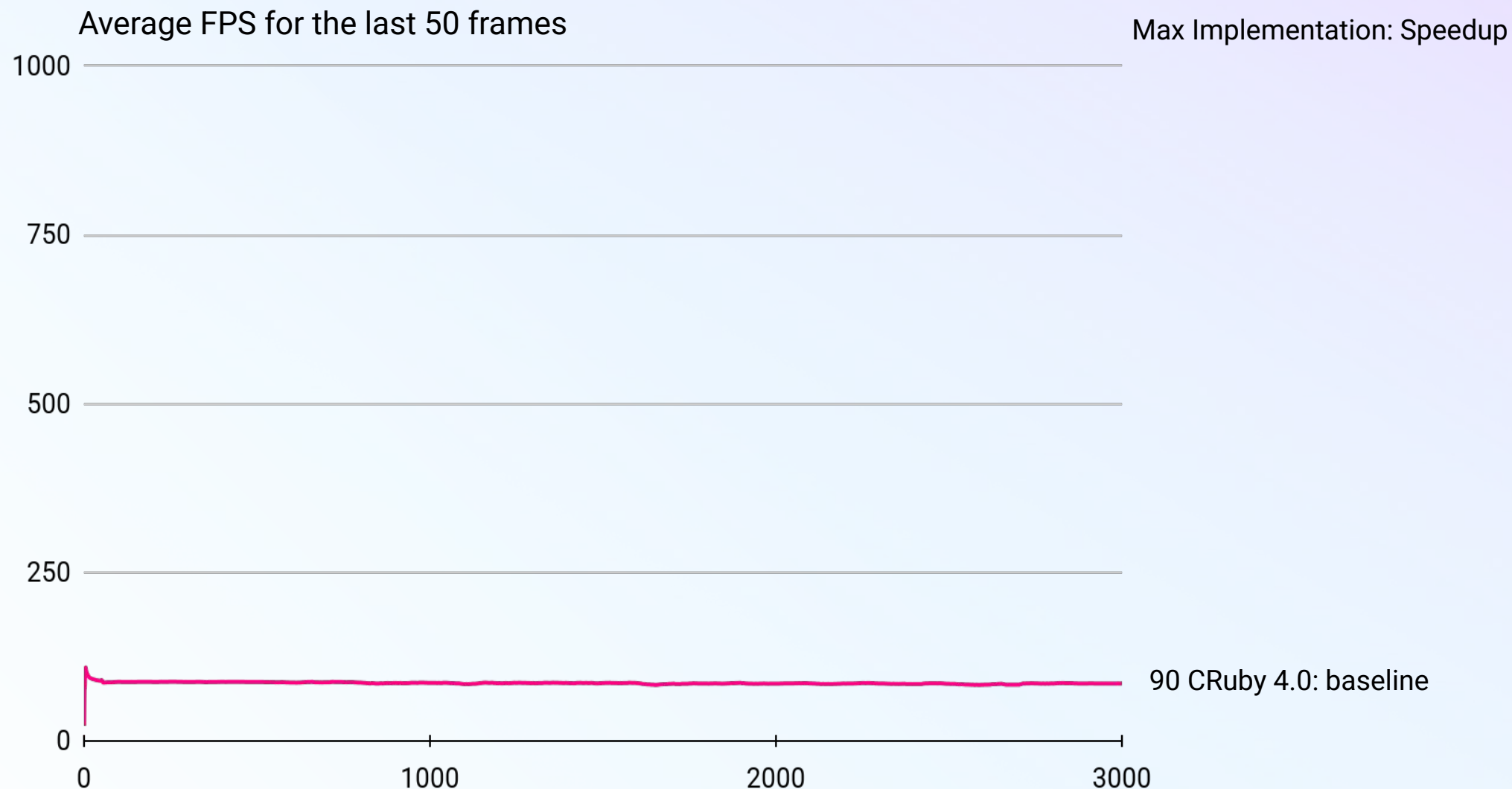


EN

From C to Ruby: Porting Doom

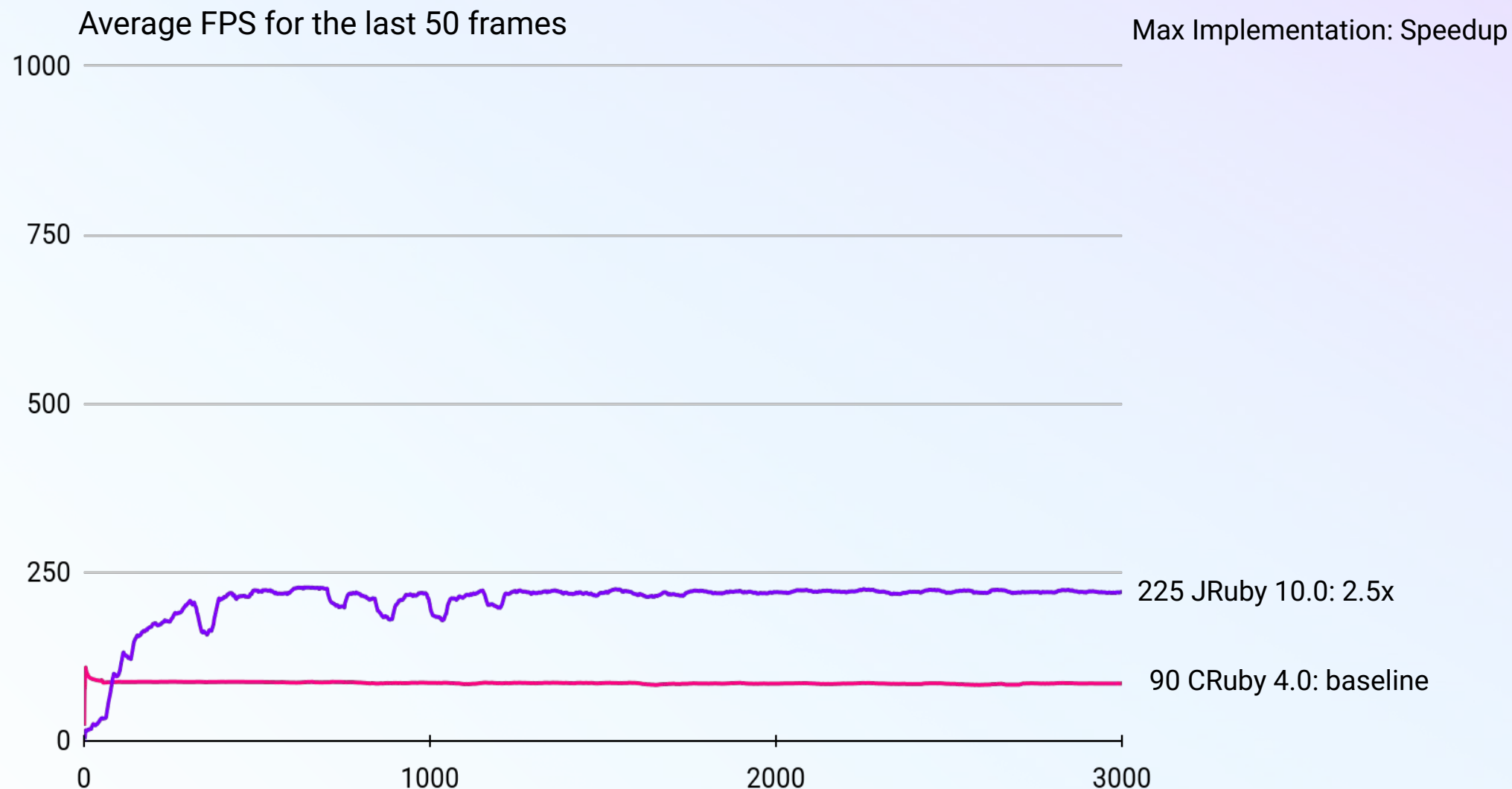
Chris Hasiński
@khasinski

OptCarrot warmup, revisited for Ruby 4.0



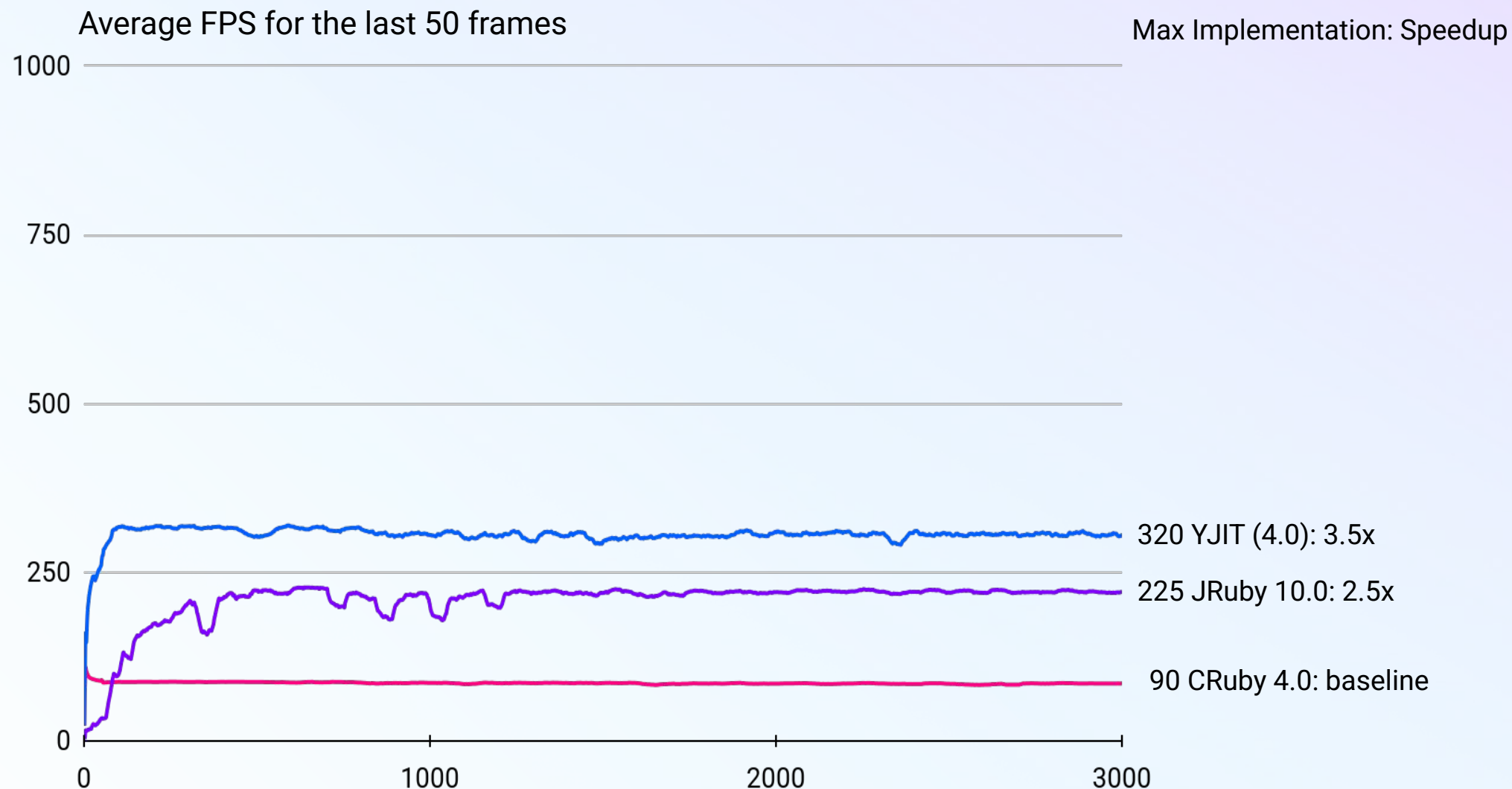
Run on this MacBook Pro M4 Max

OptCarrot warmup, revisited for Ruby 4.0



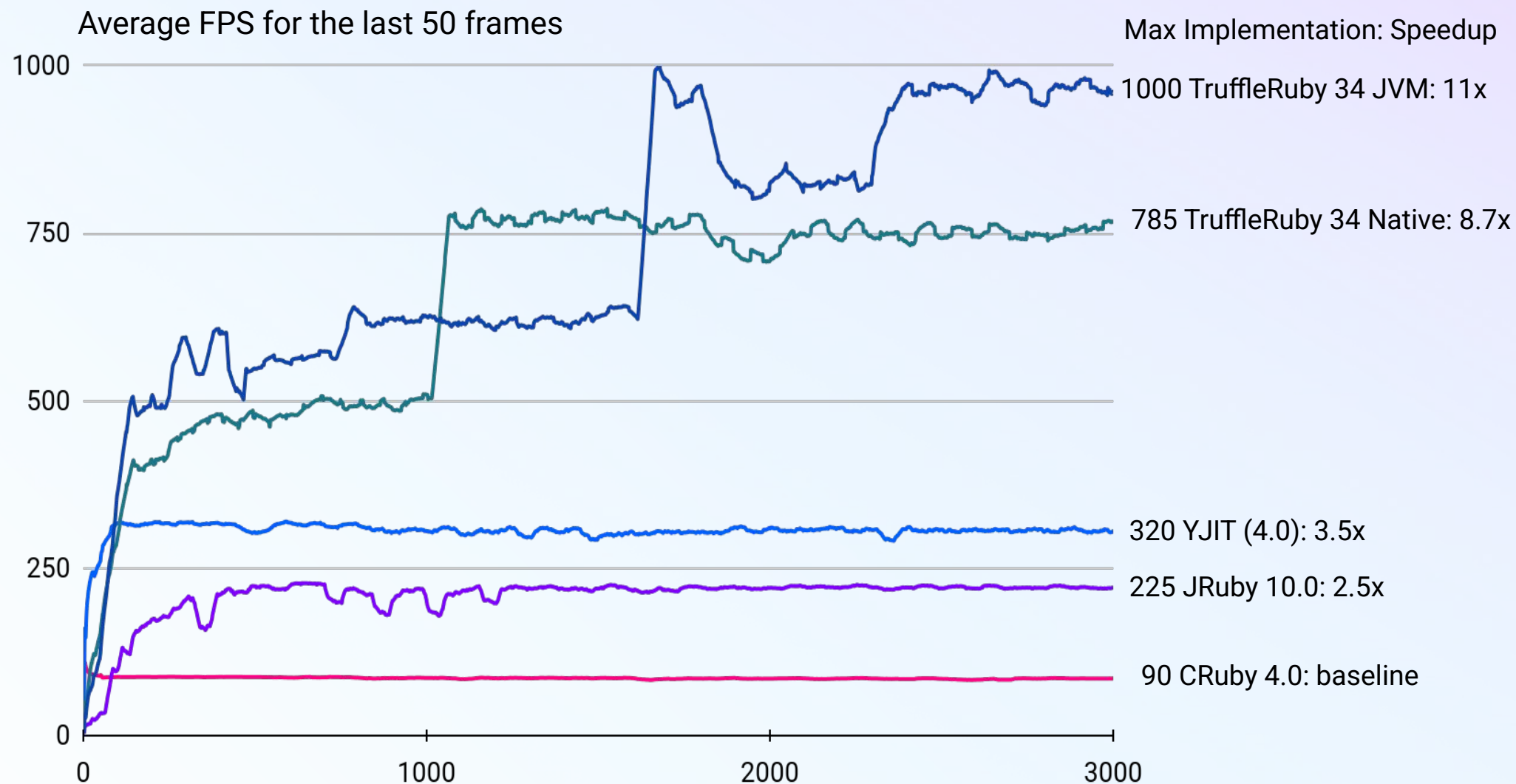
Run on this MacBook Pro M4 Max

OptCarrot warmup, revisited for Ruby 4.0



Run on this MacBook Pro M4 Max

OptCarrot warmup, revisited for Ruby 4.0



Run on this MacBook Pro M4 Max

The Challenge

How to make Hash:

- Thread-Safe
- Support parallel reads and writes
- Fast, no single-threaded overhead

Why should we make Hash Thread-Safe?

Let's try writing concurrently to a Hash:

```
hash = {}
```

```
16.times.map { |t|  
  Thread.new {  
    1000.times do |i|  
      hash[t * 1000 + i] = t  
    end  
  }  
}.each(&:join)
```

```
p hash.size
```

Why should we make Hash Thread-Safe?

CRuby 4.0.2

```
$ ruby concurrent_writes.rb
```

16000

JRuby 10.0.5.0

```
$ ruby concurrent_writes.rb
```

11612

```
$ ruby concurrent_writes.rb
```

12614

TruffleRuby 25.0.0

```
$ ruby concurrent_writes.rb
```

9884

```
$ ruby concurrent_writes.rb
```

java.lang.NullPointerException [...]

TruffleRuby 34.0.0

```
$ ruby concurrent_writes.rb
```

16000 💪

Do we really need to make Hash Thread-Safe?

No official specification that Hash must be thread-safe, but is on CRuby

I did research on thread-safe Hash in 2018 but not merged until 2026:

- When crashing it often revealed other concurrency bugs in gems
- Gems using `Concurrent::Map` work fine
- Low priority because it affected few gems at the time

Recently RubyGems and Psych started relying on Hash being thread-safe

- Cannot use concurrent-ruby there

Code relying on Hash being Thread-Safe

In RubyGems (simplified):

```
def self.stubs_for(name)
  @@stubs_by_name[name] ||= stubs_for_pattern("#{name}-*.gemspec")
end
```

In Psych (simplified):

```
DISPATCH = Hash.new do |hash, klass|
  hash[klass] = :visit_#{klass.name.gsub(':', '_')}
end.compare_by_identity

def visit(target)
  send DISPATCH[target.class], target
end
```

Can we reuse existing concurrent maps?

Java's ConcurrentHashMap

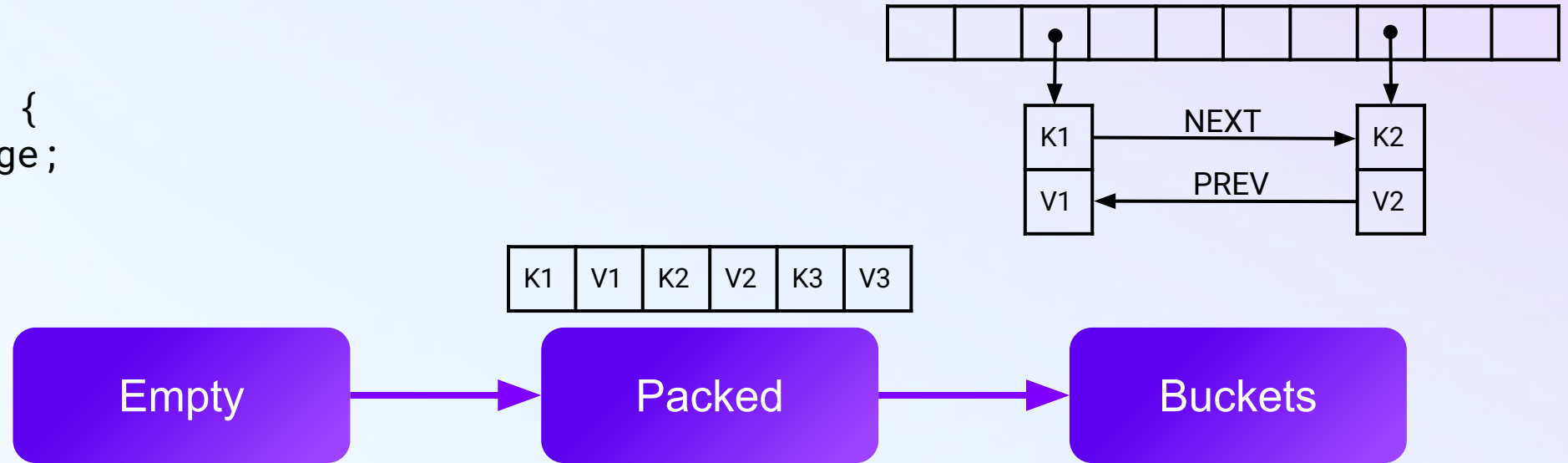
- no, does not preserve insertion ordering

Java's LinkedHashMap

- supports insertion ordering but not concurrency
- using a lock but cannot write in parallel then

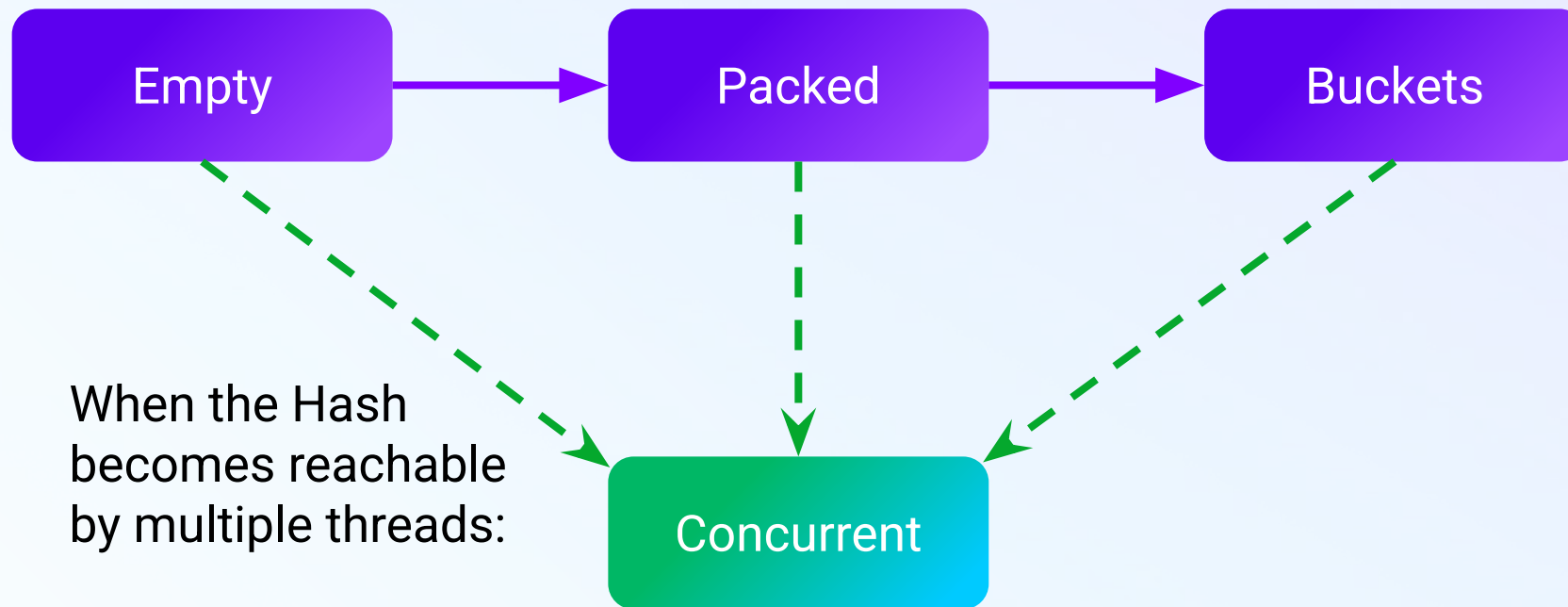
Hash Storage Strategies

```
class RubyHash {  
  Object storage;  
  int size;  
  ...  
}
```

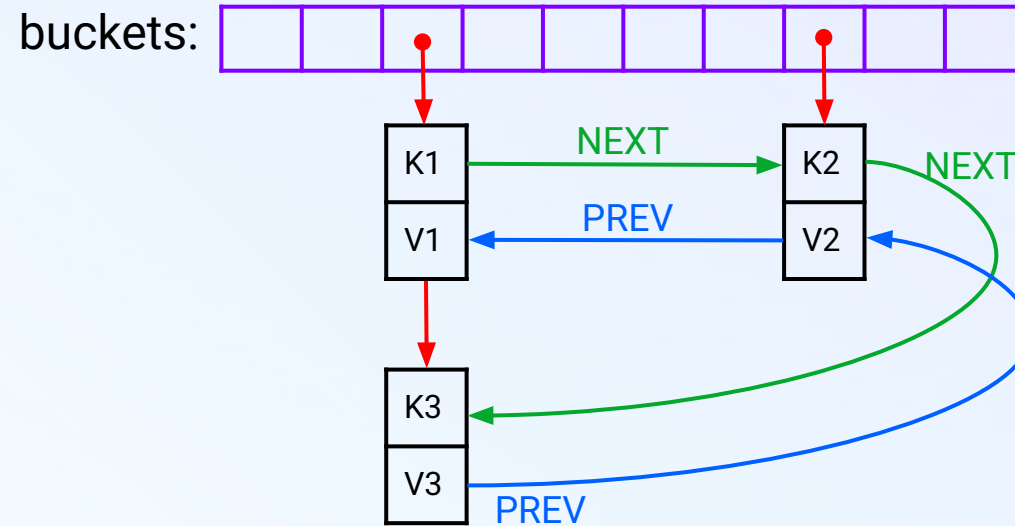


# of entries	0	1-3	4+
Thread-Safety	OK	NONE	Meh but no exceptions
Example safety issues		NullPointerException	lost writes

Concurrent Hash Storage Strategy



Concurrent Hash Storage Strategy



We have to synchronize:

- The **buckets array**
- The **singly-linked lists** in each bucket
- The **doubly-linked** list which maintains insertion order

Concurrent Hash Storage Strategy

We have to synchronize:

- The **buckets array**
 - Using a new kind of lock to support **parallel reads and writes**
- The **singly-linked lists** in each bucket
 - Lock-Free implementation based on
T. Harris. *A Pragmatic Implementation of Non-Blocking Linked-Lists*. (2001)
- The **doubly-linked** list which maintains insertion order
 - Lock-Free implementation based on
H. Sundell and P. Tsigas. *Lock-Free and Practical Doubly Linked List-Based Deques Using Single-Word Compare-and-Swap*. (2005)

Synchronizing the Buckets Array

We want:

- Parallel reads
- Parallel writes
- Exclusive locking for `Hash#compare_by_identity`

Regular Mutex/Lock

- No parallelism at all

Read-Write Lock

- No parallelism for writes

Layout Lock

- Perfect fit, new lock from N. Cohen, A. Tal, and E. Petrank.

Layout Lock: A Scalable Locking Paradigm for Concurrent Data Layout Modifications. (2017)

Paper at OOPSLA

Parallelization of Dynamic Languages: Synchronizing Built-in Collections

BENOIT DALOZE, Johannes Kepler University Linz, Austria

ARIE TAL, Technion, Israel

STEFAN MARR, University of Kent, United Kingdom

HANSPETER MÖSSENBOCK, Johannes Kepler University Linz, Austria

EREZ PETRANK, Technion, Israel

Dynamic programming languages such as Python and Ruby are widely used, and much effort is spent on making them efficient. One substantial research effort in this direction is the enabling of parallel code execution. While there has been significant progress, making dynamic collections efficient, scalable, and thread-safe is an open issue. Typical programs in dynamic languages use few but versatile collection types. Such collections are an important ingredient of dynamic environments, but are difficult to make safe, efficient, and scalable.

In this paper, we propose an approach for efficient and concurrent collections by gradually increasing synchronization levels according to the dynamic needs of each collection instance. Collections reachable only by a single thread have no synchronization, arrays accessed in bounds have minimal synchronization, and for the general case, we adopt the Layout Lock paradigm and extend its design with a lightweight version that fits the setting of dynamic languages. We apply our approach to Ruby's *Array* and *Hash* collections. Our experiments show that our approach has no overhead on single-threaded benchmarks, scales linearly for *Array* and *Hash* accesses, achieves the same scalability as Fortran and Java for classic parallel algorithms, and scales better than other Ruby implementations on Ruby workloads.

CCS Concepts: • **Software and its engineering** → **Data types and structures; Concurrent programming structures**; *Dynamic compilers*;

Additional Key Words and Phrases: Dynamically-typed languages, Collections, Concurrency, Ruby, Truffle, Graal, Thread Safety

ACM Reference Format:

Benoit Daloze, Arie Tal, Stefan Marr, Hanspeter Mössenböck, and Erez Petrank. 2018. Parallelization of Dynamic Languages: Synchronizing Built-in Collections. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 108

108

Layout Lock

3 access modes:

- Read
- Write
- Layout Change

Reads:

- Optimistic: the data is read and then a flag is checked
- If the flag indicates a layout change, retry the read

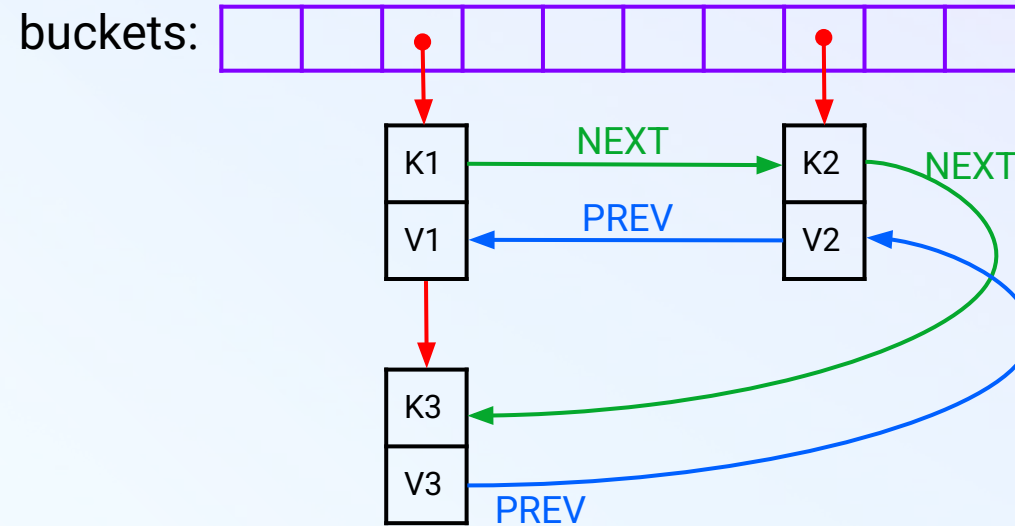
Writes:

- Block Layout Changes by setting a flag on the thread

Layout Change:

- Sets flag to invalidate reads, wait that all writes are finished and prevents new writes, performs the actual change, reset everything

Concurrent Hash Storage Strategy



We have to synchronize:

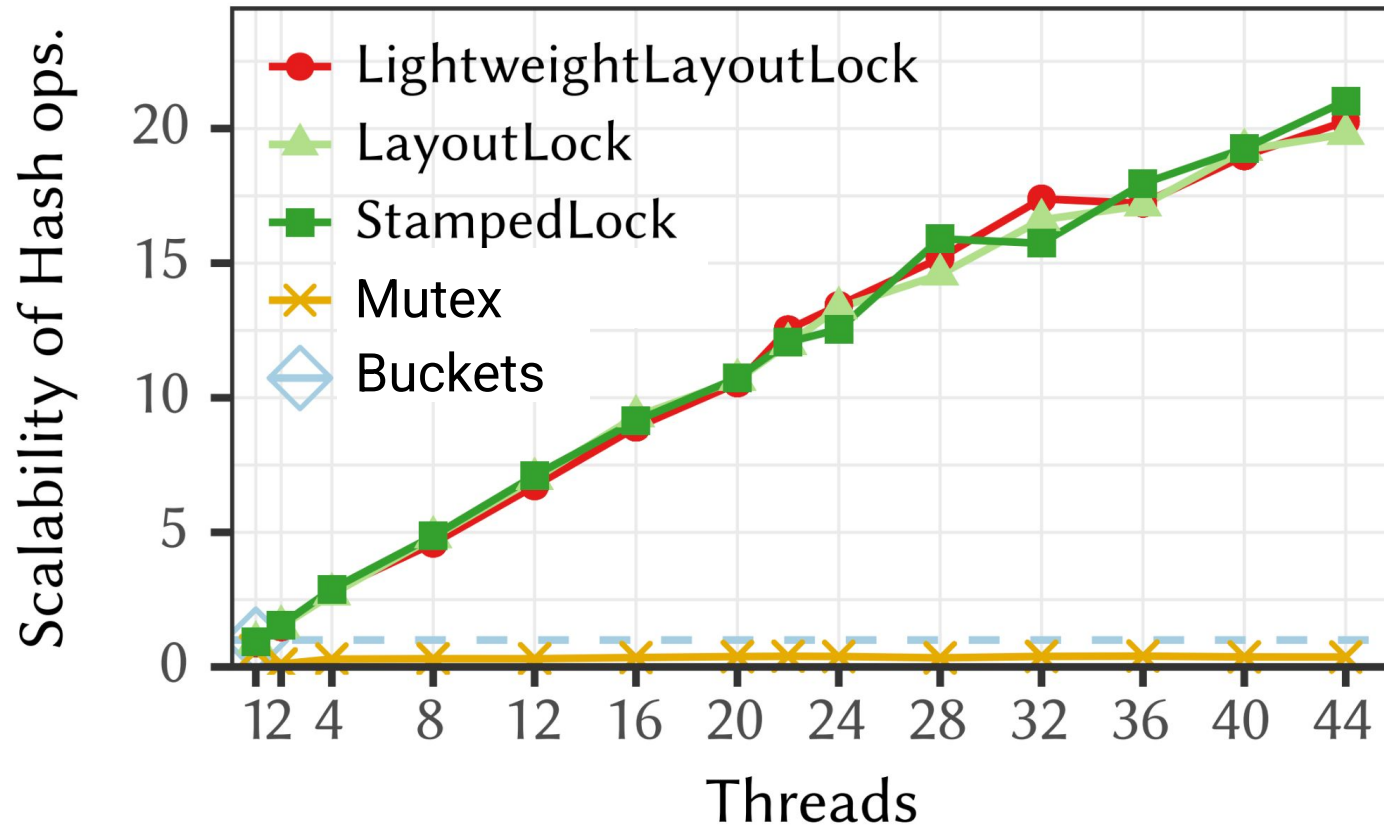
- The **buckets array**
- The **singly-linked lists** in each bucket
- The **doubly-linked** list which maintains insertion order

The Challenge

How to make Hash:

- ① Thread-Safe
 - Support parallel reads and writes
 - Fast, no single-threaded overhead

Does it scale for reads and writes?



Scalability of Hash with 65,536 keys:

- 80% reads
- 10% writes
- 10% deletes

Relative to Buckets on 1 thread

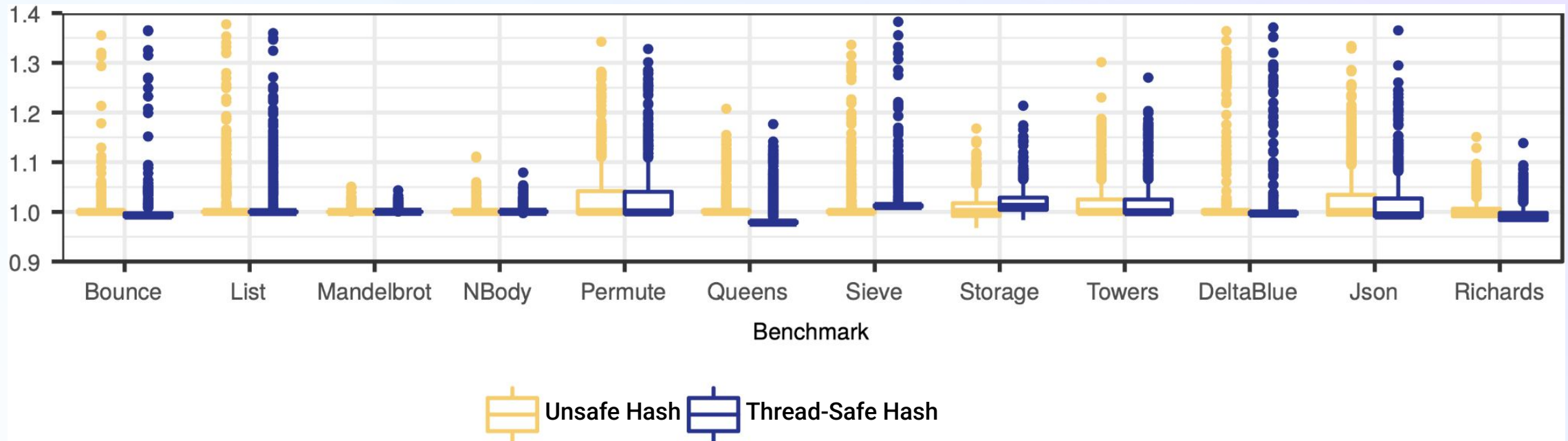
The Challenge

How to make Hash:

- ✓ Thread-Safe
- ✓ Support parallel reads and writes
 - Fast, no single-threaded overhead

Is making Hash Thread-Safe slowing it down?

Peak performance, normalized to TruffleRuby, lower is better



Zero overhead on single-threaded benchmarks
thanks to storage strategies

The Challenge

How to make Hash:

- ✓ Thread-Safe
- ✓ Support parallel reads and writes
- ✓ Fast, no single-threaded overhead

Conclusion

Hash can be made thread-safe, parallel and fast

- No need to compromise or pick 2 out of 3

TruffleRuby runs both Ruby code and native extensions in parallel

- Threads run in parallel on TruffleRuby, no global lock
- Threads are compatible with the Ruby ecosystem as-is

Ractor can be used on CRuby for parallelism but:

- more restrictive, cannot do everything Threads can do
- overhead of sending messages
- requires changing the programming model, i.e., changing code & gems

TruffleRuby: How You Can Help

Try TruffleRuby for your application, Ruby program, etc

- Let us know how it goes, whether it's good or bad

Test TruffleRuby in CI

Install TruffleRuby in 2 seconds! (it's always precompiled)

Contribute to Ruby 4.0 support:

- Fully Open-Source, you can just make a PR on GitHub
- There is a list of all Ruby 4.0 changes
- Easy: 73% of core library methods are implemented in Ruby

Kernel

- ☐ `Kernel#inspect` now checks for the existence of a `#instance_variables_to_inspect` method, allowing control over which instance variables are displayed in the `#inspect` string:

```
class DatabaseConfig
  def initialize(host, user, password)
    @host = host
    @user = user
    @password = password
  end

  private def instance_variables_to_inspect = [:@host, :@user]
end

conf = DatabaseConfig.new("localhost", "root", "hunter2")
conf.inspect #=> #<DatabaseConfig:0x0000000104def350 @host="localhost", @user="root">
```

[\[Feature #21219\]](#)

Making Hash Parallel, Thread-Safe and Fast!



Benoit Daloze



RubyKaigi



DATADOG